

APPLIANCE ENERGY PREDICTION WITH DEEP LEARNING

Multivariate Time-Series Forecasting of
Household Appliance Energy Consumption

Deep Learning AI Assessment

Sachini P. Nethranjali

October 2026

Source code:

<https://github.com/NethranjaliSE/multivariate-energy-forecasting>

Live demo:

<https://huggingface.co/spaces/NethranjaliSE/appliance-energy-forecasting>

Trained model:

<https://huggingface.co/NethranjaliSE/appliance-energy-cnn-lstm>

Table of Contents

1	Introduction	1
1.1	Objectives	1
1.2	Summary of Results	1
2	Data Insights	2
2.1	Dataset Description	2
2.2	Data Quality	2
2.3	Trends and Seasonal Patterns	2
2.4	Correlations	4
2.5	Autocorrelation	5
3	Preprocessing	5
3.1	Handling Missing Values	5
3.2	Outlier Detection and Treatment	5
3.3	Scaling and Normalisation	6
3.4	Data Splitting	6
3.5	Preventing Data Leakage	7
4	Feature Engineering	7
4.1	Created Features	7
4.2	Choosing Lags from Autocorrelation	8
4.3	Feature Selection	9
5	Model Design	10
5.1	Baseline Models	10
5.2	Input Representation	10
5.3	Architectures	11
5.4	Activation Functions	11
5.5	Optimiser and Loss Function	11
5.6	Regularisation and Model Complexity	12
5.7	Implementation	12
6	Results	12
6.1	Baselines	12
6.2	LSTM	13
6.3	Comparing Architectures	13
6.4	Final Model	14
6.5	Error Analysis	15
6.6	Comparison with Published Work	15

7	Model Optimisation	16
7.1	Hyperparameter Tuning with Random Search	16
7.2	Dropout Experiment	17
7.3	Early Stopping	18
7.4	Seed Ensemble	18
7.5	Performance Before and After Optimisation	18
8	Challenges and Solutions	19
8.1	A Strong Naive Baseline	19
8.2	A Skewed Target and Slow Training	19
8.3	Avoiding Data Leakage	19
8.4	Telling Real Improvements from Luck	19
8.5	Reporting Precision	19
8.6	Deploying the Model	20
9	Conclusion	20
9.1	Summary	20
9.2	Key Findings	20
9.3	Limitations	20
9.4	Future Work	21
	References	22

List of Figures

Figure 1	Appliance energy use over the whole period and averaged by hour of day	3
Figure 2	Average appliance energy use by day of week and hour of day .	3
Figure 3	Daily average consumption with its 7-day rolling average	4
Figure 4	Correlation matrix of all variables	4
Figure 5	Autocorrelation of appliance consumption for lags up to 48 hours	5
Figure 6	Target distribution, box plot with IQR outliers, and the log-transformed target	6
Figure 7	Chronological training, validation and test split	7
Figure 8	Random Forest feature importance for the top 15 features	9
Figure 9	LSTM training and validation loss, and test predictions for the first three days	13
Figure 10	Test error of every model, and final-model predictions for the first three test days	14
Figure 11	Residuals of the final model and its predictions against actual values	14
Figure 12	Final-model error by hour of day and by actual consumption level	15
Figure 13	Architecture comparison and dropout experiment, with spread across seeds	18

List of Tables

Table 1	Chronological split of the data	6
Table 2	Engineered features and the reasoning behind them	8
Table 3	Autocorrelation at candidate lags and the decision for each	9
Table 4	Validation error of a Random Forest using the top-ranked features	10
Table 5	Baseline results on the test set	12
Table 6	Architecture comparison on the validation set, averaged over three seeds	13
Table 7	Final comparison of all models on the test set	14
Table 8	Published test results on the same dataset	16
Table 9	Random-search space for the CNN-LSTM	16
Table 10	Top search configurations retrained with three seeds (validation MAE in Wh)	17
Table 11	Dropout experiment on the CNN-LSTM, averaged over three seeds	17
Table 12	CNN-LSTM test performance before and after optimisation	18

1 Introduction

Appliances account for a large and unpredictable share of the electricity a household uses. Lighting and heating follow fairly regular patterns, but appliance use depends on what the occupants happen to be doing: someone starts the washing machine, puts something in the oven, or leaves for the weekend. Being able to anticipate this demand, even a few minutes ahead, is useful for home energy management systems, for shifting flexible loads and for spotting unusual consumption early. Deep learning has become a common tool for this kind of forecasting in buildings [3].

In this project I worked with the *Appliances Energy Prediction* dataset [1, 2], which records the energy used by appliances in a Belgian house every 10 minutes, together with indoor temperature and humidity readings and local weather. I framed the problem as **one-step-ahead forecasting**: given everything known up to a particular moment, predict how much energy the appliances will use in the next 10-minute interval. This follows the brief’s request to predict future consumption from past values, and it means the model may only ever see information from before the interval it is predicting.

1.1 Objectives

The work had four goals:

1. Explore the data and prepare it without letting any future information leak into training.
2. Build time-based, lagged, rolling, interaction and domain features, and keep only the useful ones.
3. Set up simple baseline models, then design, compare and tune several deep learning architectures.
4. Evaluate everything on a later, unseen period using MAE, RMSE, MAPE and R^2 .

1.2 Summary of Results

The final model is an ensemble of three CNN-LSTM networks whose forecasts are averaged. On the test period it reaches a **mean absolute error of 22.80 Wh** and a **root mean squared error of 55.71 Wh**, with $R^2 = 0.60$. For comparison, simply repeating the last reading, which is a surprisingly strong forecast at this short horizon, gives 26.05 Wh and 65.19 Wh. The final model therefore cuts the average error by 12.5% and the root mean squared error by 14.5%, and it also beats Linear Regression, a Random Forest and a single LSTM. All code is available in the project repository [10].

2 Data Insights

2.1 Dataset Description

The dataset holds 19,735 readings taken every 10 minutes between 11 January and 27 May 2016, which is roughly four and a half months. The target variable, **Appliances**, is the energy used by appliances in each interval, measured in watt-hours. Alongside it, the file contains:

- **lights**, the energy used by light fittings;
- temperature and relative humidity from nine areas of the house (T1–T9, RH_1–RH_9), where sensor 6 is mounted outside the building;
- weather from the nearest airport: temperature, humidity, pressure, wind speed, visibility and dew point;
- two columns of random numbers (**rv1**, **rv2**) that the dataset’s authors added on purpose, to check whether a feature selection method can tell real signal from noise.

The brief also mentions **NSM**, **WeekStatus** and **Day_of_week**. These were not in the file I downloaded, so I created them from the timestamp during feature engineering.

2.2 Data Quality

Before any analysis, I checked the data for problems. There were **no missing values**, **no duplicate timestamps** and **no gaps**: every reading is exactly 10 minutes after the previous one. The target, however, is very unevenly distributed. Its median is 60 Wh and three quarters of all readings are at or below 100 Wh, yet the mean is pulled up to 97.7 Wh by occasional readings as high as 1,080 Wh (Figure 6).

2.3 Trends and Seasonal Patterns

Plotting the whole series shows a quiet baseline of about 50 Wh broken up by frequent, short bursts of several hundred watt-hours (Figure 1). There are also two stretches, in late January and around 1 April, where consumption stays flat for days. My reading is that the occupants were away during those periods.

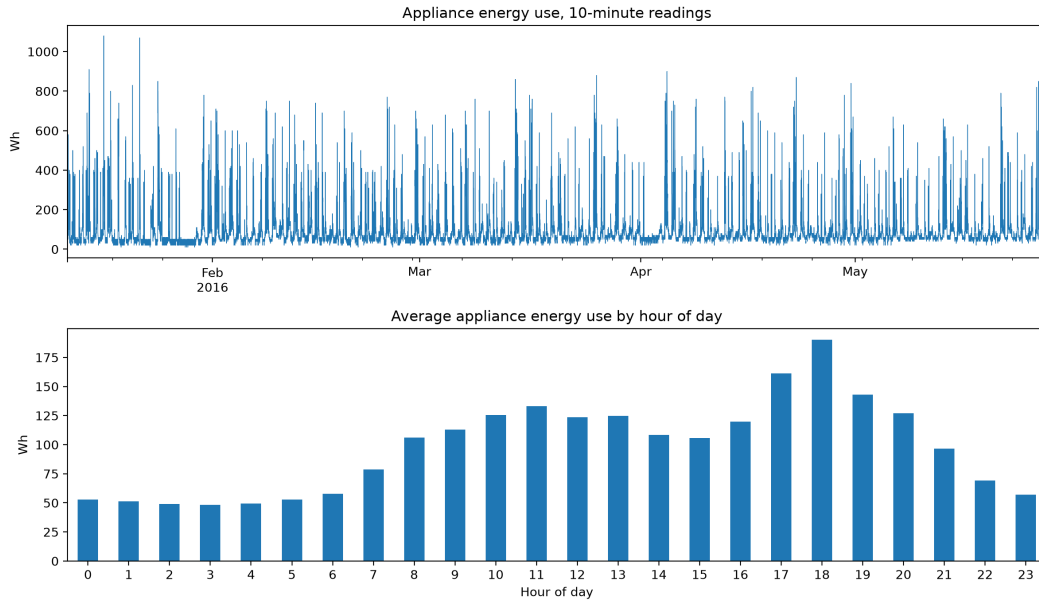


Figure 1: Appliance energy use over the whole period and averaged by hour of day

Three patterns stand out:

- **A strong daily rhythm.** Overnight use sits around 50 Wh. It starts climbing at about 07:00 and peaks between 17:00 and 18:00, when it is close to four times the night-time level.
- **A weak weekly rhythm.** The evening peak shows up on every day of the week, and on Sundays activity simply starts a little later (Figure 2). The median reading is between 60 and 70 Wh on every weekday, so the day of the week matters much less than the hour.
- **No long-term trend.** Once the noise is smoothed out with daily and 7-day averages, consumption stays between roughly 80 and 120 Wh for the whole period (Figure 3). Four and a half months is also far too short to contain any yearly seasonality.

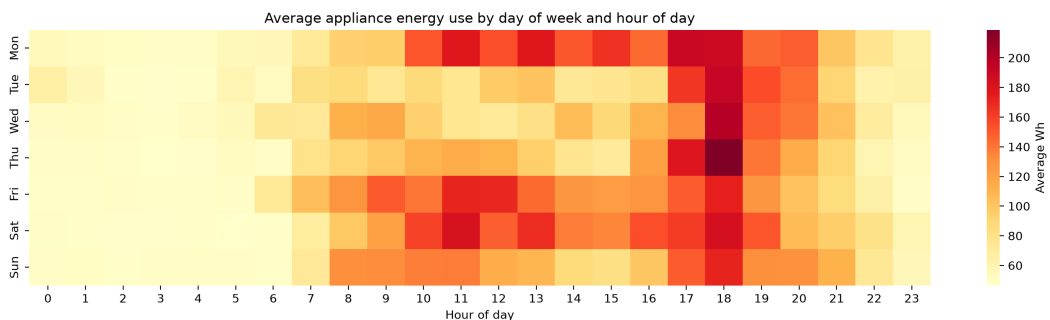


Figure 2: Average appliance energy use by day of week and hour of day

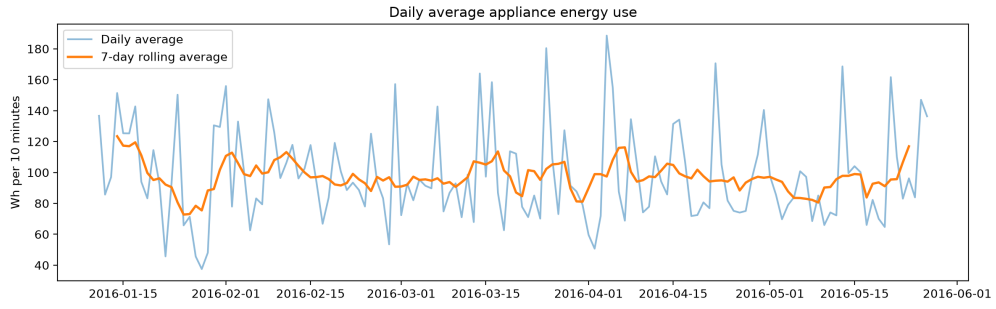


Figure 3: Daily average consumption with its 7-day rolling average

2.4 Correlations

I expected the indoor sensors to say a lot about appliance use, for example a warm, humid kitchen during cooking. The correlation matrix in Figure 4 shows that this is not the case. The strongest correlation with Appliances is lights, at only 0.20, and every other variable falls below 0.13 in absolute value; outdoor humidity is the most negative at -0.15 . The indoor temperatures are strongly correlated with each other, and the outdoor sensor T6 almost duplicates the airport temperature T_out, so many columns carry overlapping information. As expected, the two random columns are uncorrelated with the target (-0.011) and are identical to each other.

The practical conclusion was that the current sensor readings on their own would not get a model very far. Past consumption and the time of day looked like much more promising sources of information, and that shaped the feature engineering.

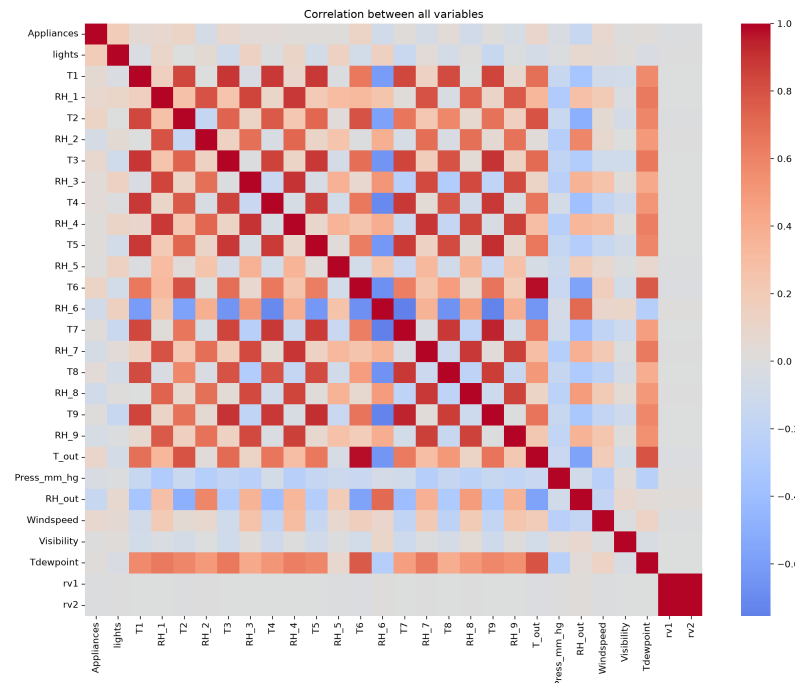


Figure 4: Correlation matrix of all variables

2.5 Autocorrelation

Autocorrelation measures how closely consumption at one moment follows consumption a given number of steps earlier. Figure 5 shows a strong short-term memory that fades over the first hour, a plateau between one and two hours, almost nothing at six to twelve hours, and then a second peak at exactly 24 hours, which is the daily routine repeating itself. I used these values directly to choose the lag features (Section 4).

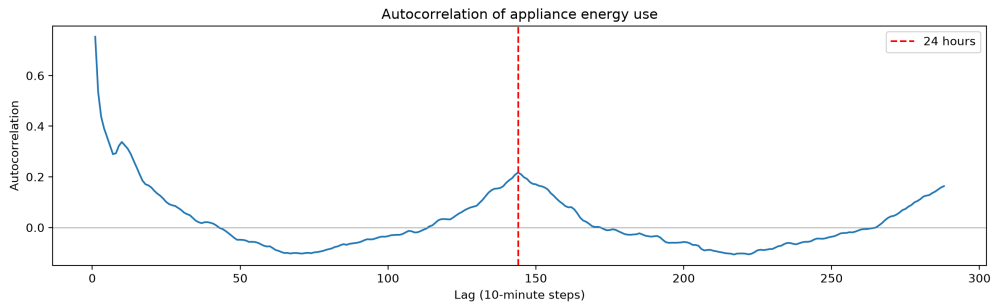


Figure 5: Autocorrelation of appliance consumption for lags up to 48 hours

3 Preprocessing

3.1 Handling Missing Values

Because the checks found no missing values, duplicates or gaps, no imputation or deletion was needed. I kept the checks in the notebook anyway, so that a reader can see the data was verified and not just assumed to be complete.

3.2 Outlier Detection and Treatment

I used the interquartile range rule to find outliers. With the first and third quartiles at 50 and 100 Wh, the upper limit is $Q_3 + 1.5 \times \text{IQR} = 175$ Wh, and **2,138 readings, or 10.8% of the data**, lie above it (Figure 6).

Deciding what to do with them was one of the more important choices in the project. One reading in nine is far too many to blame on faulty sensors. These are real moments of heavy use, such as cooking or laundry, and they are exactly the events a forecasting model should try to capture. Removing or capping them would have trained the model to underestimate consumption. I therefore **kept every outlier** and instead reduced their influence by modelling the target on a logarithmic scale, $\log(1 + \text{Appliances})$. The right-hand panel of Figure 6 shows that this transform makes the distribution far less skewed. Every prediction is converted back to watt-hours before it is evaluated, so all reported errors are in the original units.

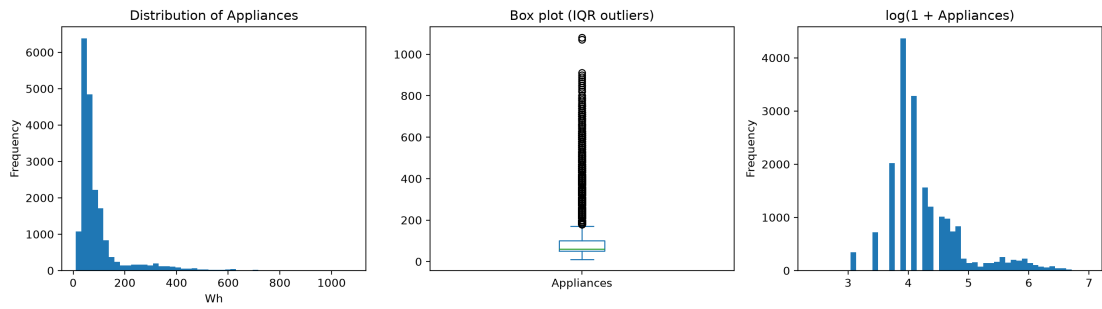


Figure 6: Target distribution, box plot with IQR outliers, and the log-transformed target

3.3 Scaling and Normalisation

I **standardised** every feature to zero mean and unit variance. Min-max scaling was the obvious alternative, but with spikes reaching 1,080 Wh it would have squeezed most ordinary values into a tiny part of the range. Standardisation also tends to make gradient-based training of neural networks more stable. The scaler was fitted on the training data only and then applied unchanged to the validation and test data. The log-transformed target was standardised in the same way, which turned out to matter a great deal (Section 8).

3.4 Data Splitting

Shuffling rows before splitting is common in machine learning, but it would be a mistake here: the model would be trained on days on both sides of each test reading. I split the data **in time order** instead (Table 1, Figure 7). The first 70% is used for training, the next 10% for validation, which drives early stopping and every modelling decision, and the final 20% for testing. Training and validation together make up the first 80% requested in the brief. The test period was touched only for final evaluations, never to choose between models or settings.

Table 1: Chronological split of the data

Part	Rows	Period
Training	13,713	12 January – 16 April 2016
Validation	1,959	16 April – 30 April 2016
Test	3,919	30 April – 27 May 2016

The first 24 hours of data are dropped before splitting, because the 24-hour lag feature has no history for them.

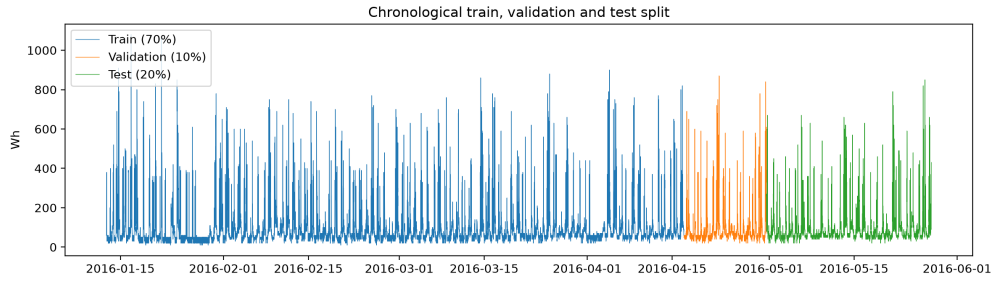


Figure 7: Chronological training, validation and test split

3.5 Preventing Data Leakage

A forecast is only honest if the model cannot see anything from the interval it is predicting. To guarantee this, I **shifted every sensor reading and every consumption-based feature back by at least one step**, so the features for time t only use readings up to $t - 1$. Rolling statistics are calculated on the already shifted series. I also tested this directly: changing the target or a sensor reading at time t leaves every feature at time t unchanged. Together with the chronological split and training-only scaling, this rules out leakage from the future. Finally, I dropped the two random columns, `rv1` and `rv2`.

4 Feature Engineering

4.1 Created Features

Starting from the cleaned data, I built 51 candidate features, summarised in Table 2. The code lives in `src/feature_engineering.py`.

Table 2: Engineered features and the reasoning behind them

Group		Features	Reasoning
Lagged consumption		Consumption 10, 20, 30, 60 and 120 minutes ago and 24 hours ago; the latest change	Chosen from the autocorrelation (Table 3)
Rolling statistics		Mean, standard deviation and maximum over the past 1 hour and 3 hours	Smooth out noise and describe how volatile use has recently been
Time		Seconds since midnight (NSM), hour as sine and cosine, day of week, weekend flag	Capture the daily cycle; the sine and cosine pair keeps 23:50 and 00:00 close together
Interaction		Indoor–outdoor temperature gap; temperature $\times$ humidity; kitchen humidity above the house average; weekend $\times$ hour	Heating demand, cooking activity, and a different daily shape at weekends
Domain		Belgian public holidays	Holidays change household routines
Original readings	read-	Lighting energy, indoor and outdoor temperature and humidity, weather	Kept as candidates for selection

4.2 Choosing Lags from Autocorrelation

The brief asks for lag periods based on autocorrelation, and Table 3 shows how each choice was made. The lags at 10, 20 and 30 minutes and at one and two hours cover the strong short-term memory and the plateau. The 24-hour lag captures the daily routine. Lags of six and twelve hours were left out because their autocorrelation is essentially zero, and the three-hour window is already covered by the rolling statistics.

Table 3: Autocorrelation at candidate lags and the decision for each

Lag	Autocorrelation	Decision
10 minutes	0.753	used
20 minutes	0.534	used
30 minutes	0.438	used
1 hour	0.324	used
2 hours	0.311	used
3 hours	0.171	covered by the 3-hour rolling statistics
6 hours	0.021	not used
12 hours	-0.101	not used
24 hours	0.217	used (daily routine)

I deliberately left out a *month* feature. With only four and a half months of data, the test period falls mostly in May, a month the model would barely have seen during training, so the feature would have been more likely to mislead than to help.

4.3 Feature Selection

To find out which of the 51 features actually mattered, I trained a Random Forest on all of them and ranked them by importance (Figure 8). One feature dominates: the reading from ten minutes earlier (`app_lag_1`) accounts for about 78% of the total importance. I then trained Random Forests on the top 1, 5, 10, 15, 20, 30 and all 51 features and compared them on the **validation set** (Table 4).

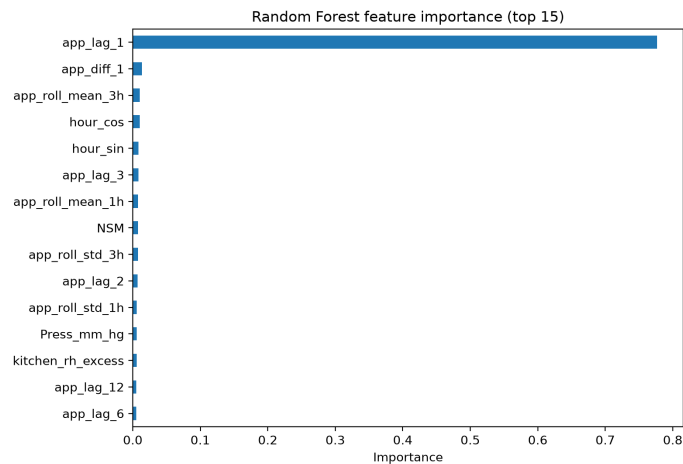
**Figure 8:** Random Forest feature importance for the top 15 features

Table 4: Validation error of a Random Forest using the top-ranked features

Number of features	Validation MAE (Wh)	Validation RMSE (Wh)
1	25.03	64.11
5	23.98	57.95
10	23.37	58.11
15	23.64	59.08
20	23.97	59.62
30	24.52	59.49
51	24.10	59.43

The **top 10 features** gave the lowest validation error, and adding the other 41 made results slightly worse rather than better. The ten selected features are consumption 10, 20 and 30 minutes earlier, the latest change, the one-hour and three-hour rolling means, the three-hour rolling standard deviation, seconds since midnight, and the sine and cosine of the hour. Every one of them describes either recent consumption or the time of day. None of the temperature or humidity sensors made the cut, which agrees with their weak correlations, and the weekend and holiday flags ranked last, which agrees with the weak weekly pattern seen earlier.

5 Model Design

5.1 Baseline Models

Before building anything complex, I set up three baselines to give the deep learning models something concrete to beat:

- a **naive forecast**, which predicts that the next reading will equal the last one;
- **Linear Regression** on all 51 standardised features;
- a **Random Forest** with 200 trees and at least five samples per leaf.

The naive forecast is easy to dismiss, but at a ten-minute horizon it is hard to beat, because consumption rarely changes much from one interval to the next.

5.2 Input Representation

The recurrent models take sequences rather than single rows. Each input is the **last 12 steps, or two hours, of the 10 selected features**, so its shape is 12×10 . Two

hours covers the plateau in the autocorrelation. I also tried windows of one hour, which performed worse on the validation set, and four hours, which added nothing.

5.3 Architectures

I compared four architectures. They share the same output layers and the same training procedure, so any difference between them comes from the part that processes the sequence:

- **LSTM:** a single long short-term memory layer with 64 units [7]. Its gates learn which parts of the recent past are worth remembering.
- **GRU:** a single gated recurrent unit layer with 64 units. It is a simpler cell with about a quarter fewer parameters than the LSTM.
- **CNN-LSTM:** a one-dimensional convolution (32 filters, kernel size 3) followed by an LSTM with 64 units, following the hybrid design of Kim and Cho [6]. The convolution picks up short local patterns spanning about 30 minutes, such as the start of a spike, and the LSTM then models how those patterns develop. I used causal padding so that the convolution cannot look ahead within the window.
- **Stacked LSTM:** two LSTM layers with 64 and 32 units and dropout in between, to test whether a deeper network helps.

The final architecture, which has 27,937 parameters per network, is:

```
Conv1D(32, 3, ReLU) → LSTM(64, tanh) → Dropout(0.3) → Dense(32, ReLU)
                               → Dense(1, linear)
```

5.4 Activation Functions

- **tanh** inside the recurrent layers. It is the standard choice for LSTMs, keeps the internal state within a bounded range, and allows TensorFlow to use its fast implementation.
- **ReLU** in the convolutional and hidden dense layers. It adds non-linearity without saturating, which helps gradients flow during training.
- A **linear** output layer, because the model predicts a continuous quantity.

5.5 Optimiser and Loss Function

I trained every network with **Adam** [8], using a learning rate of 0.001 and a batch size of 64. Adam adapts the step size for each weight on its own, which makes it reliable without

much tuning. I also tried learning rates of 0.0003 and 0.003 during the hyperparameter search. The loss function is **mean squared error** on the standardised log target, the usual choice for regression, which penalises large mistakes heavily.

5.6 Regularisation and Model Complexity

Several measures keep the models from overfitting:

- **Dropout** before the hidden dense layer, with the rate chosen by experiment (Section 7).
- **Early stopping** on the validation loss with a patience of 10 epochs, restoring the best weights seen.
- **Learning-rate reduction**, which halves the learning rate after five epochs without improvement.
- **Small networks** of roughly 17,000 to 33,000 parameters, against about 13,700 training sequences.

5.7 Implementation

I built the models in TensorFlow 2.21 using the Keras interface. Its built-in training loop and callbacks for early stopping and learning-rate scheduling keep the code short, which leaves less room for mistakes. Each model trains in about a minute on an ordinary laptop processor, so no GPU was needed. The reusable code is split into modules for data preprocessing, feature engineering, model definitions, training and prediction, and the notebook imports these modules for the analysis and experiments [10].

6 Results

6.1 Baselines

Table 5: Baseline results on the test set

Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	R^2
Naive (last value)	26.05	65.19	21.86	0.45
Linear Regression	27.26	66.49	22.12	0.43
Random Forest	26.51	58.30	23.19	0.56

As Table 5 shows, neither Linear Regression nor the Random Forest managed to beat the naive forecast on MAE. The Random Forest did improve RMSE by about 10%, but a look at its predictions shows why it struggles: it mostly repeats the last reading and follows every spike one step behind. This set a clear target for the deep learning models: an MAE below 26 Wh and an RMSE below 58 Wh.

6.2 LSTM

The single LSTM was the first model to beat the naive forecast on MAE, reaching **23.41 Wh**, with an RMSE of **56.78 Wh** (Figure 9). Early stopping ended training after 36 epochs. Both loss curves level off and the validation loss never starts to rise, so the model was not overfitting. The validation loss sits below the training loss throughout, which looks odd at first but has two simple causes: dropout is only active during training, and the validation weeks happen to be quieter than the training months.

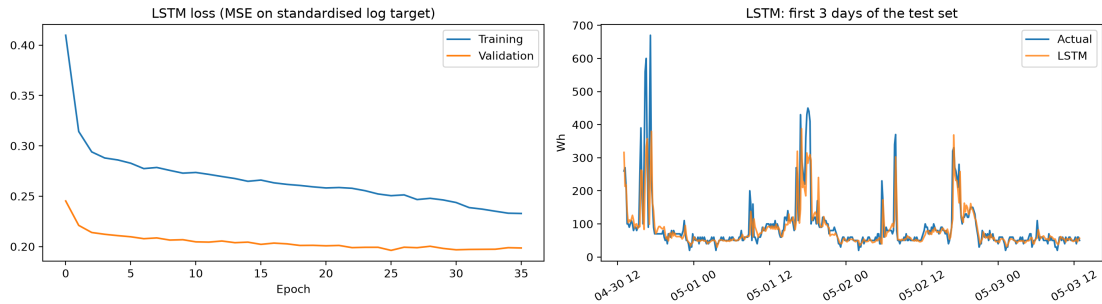


Figure 9: LSTM training and validation loss, and test predictions for the first three days

6.3 Comparing Architectures

Training the same network twice with different random seeds gives slightly different results, and early on I noticed that these differences could be as large as the differences between architectures. I therefore trained each architecture with **three seeds** and compared the averages (Table 6).

Table 6: Architecture comparison on the validation set, averaged over three seeds

Model	Parameters	Val. MAE (Wh)	Spread (std)	Val. RMSE (Wh)
LSTM	21,313	23.61	0.27	59.53
GRU	16,705	24.13	0.42	59.91
CNN-LSTM	27,937	23.05	0.14	58.33
Stacked LSTM	32,705	23.48	0.31	59.42

The **CNN-LSTM** came out ahead on both error measures and was also the most consistent from one seed to the next, so I carried it forward. The stacked LSTM has 50% more parameters than the single LSTM but did no better, which suggests that extra depth is not useful for a dataset of this size.

6.4 Final Model

Table 7 brings all the models together on the test set, and Figures 10 and 11 show how the final model behaves.

Table 7: Final comparison of all models on the test set

Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	R^2
Naive (last value)	26.05	65.19	21.86	0.45
Linear Regression	27.26	66.49	22.12	0.43
Random Forest	26.51	58.30	23.19	0.56
LSTM	23.41	56.78	19.38	0.58
CNN-LSTM, before optimisation	23.70	57.55	19.72	0.57
CNN-LSTM ensemble (final)	22.80	55.71	19.01	0.60

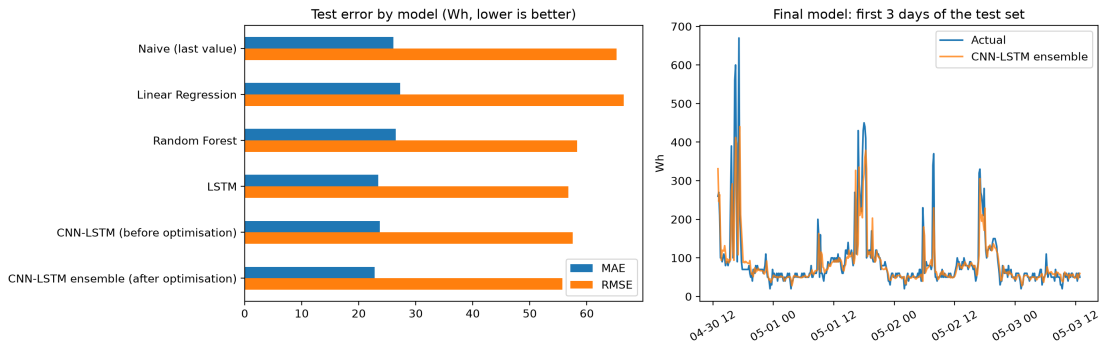


Figure 10: Test error of every model, and final-model predictions for the first three test days

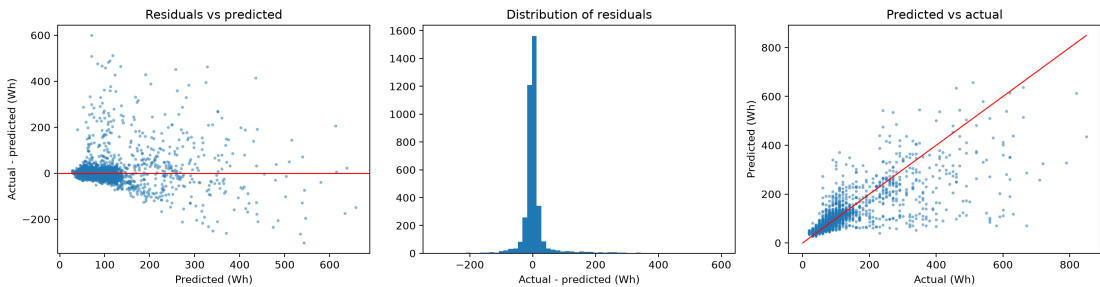


Figure 11: Residuals of the final model and its predictions against actual values

Against the naive forecast, the final model lowers MAE from 26.05 to 22.80 Wh, a reduction of 12.5%, and RMSE from 65.19 to 55.71 Wh, a reduction of 14.5%. R^2 rises from 0.45 to 0.60. The gain is larger for RMSE because RMSE weighs big misses more heavily, and big misses around spikes are exactly where repeating the last value goes most wrong.

One result surprised me. On the test set, the single CNN-LSTM (23.70 Wh) did slightly worse than the single LSTM (23.41 Wh), even though it had been better on validation. A difference of 0.3 Wh is within the variation I had already seen between seeds, so I do not read much into it. It is a good illustration of why the final model averages several networks: a single run, of either architecture, is not a reliable basis for a fine comparison.

6.5 Error Analysis

The residual plots in Figure 11 show where the remaining error comes from. Errors are small around the everyday baseline and grow as consumption rises. In the predicted-versus-actual plot the points follow the diagonal up to about 200 Wh and then drop below it, and the residuals have a long positive tail. In other words, the model tends to **underestimate large spikes**. Figure 12 breaks the error down further. Errors are largest during the morning and evening peaks. Readings below 100 Wh make up 73.9% of the test period and are predicted closely, while readings above 400 Wh make up only 2.3% of the period but produce by far the largest errors.

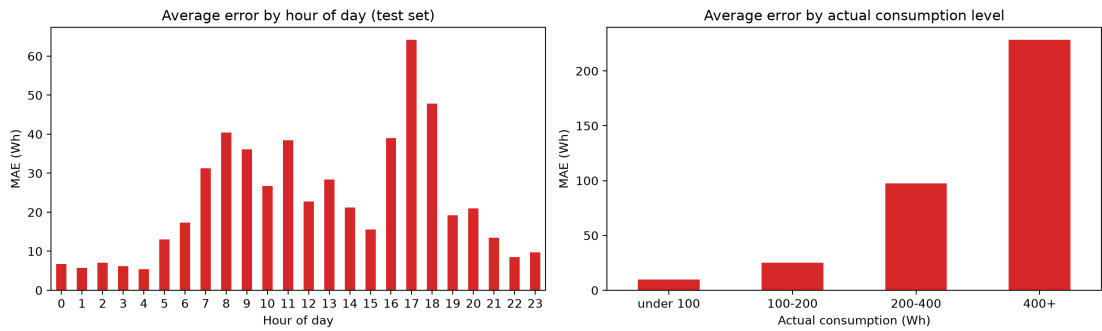


Figure 12: Final-model error by hour of day and by actual consumption level

6.6 Comparison with Published Work

Several earlier studies used the same dataset (Table 8).

Table 8: Published test results on the same dataset

Study	Model	MAE (Wh)	RMSE (Wh)
Candanedo et al. (2017) [1]	Gradient boosting	35.22	66.65
Suranata et al. (2021) [4]	LSTM	26.98	62.01
Assadian & Assadian (2023) [5]	Extra Trees	26.62	59.61
This project	CNN-LSTM ens.	22.80	55.71

These numbers are a useful reference point, but they should not be read as a direct ranking. The earlier studies did not report a chronological split, and they estimated consumption from sensor readings taken during the same interval, without using past consumption. My model forecasts the next interval and is tested on a later period it never saw. That is the stricter setting, and the lag features give it most of its advantage.

7 Model Optimisation

7.1 Hyperparameter Tuning with Random Search

The search space for the CNN-LSTM, listed in Table 9, contains 8,748 possible combinations. Training all of them was out of the question, so I used **random search** and evaluated 20 randomly chosen configurations on the validation set. Random search is a sensible choice for wide spaces like this, because typically only a few of the hyperparameters have a real effect on the result [9].

Table 9: Random-search space for the CNN-LSTM

Hyperparameter	Values tried
Window length (steps)	6, 12, 24
Convolution filters	16, 32, 64
Kernel size	2, 3, 5
LSTM units	32, 64, 128
Dense units	16, 32, 64
Dropout rate	0.1, 0.2, 0.3, 0.4
Learning rate	0.0003, 0.001, 0.003
Batch size	32, 64, 128

The best single run used a four-hour window, 16 filters, a kernel size of 5, 64 LSTM

units, dropout of 0.2 and a learning rate of 0.003, and reached a validation MAE of 22.94 Wh. That looked promising. However, when you pick the best of 20 noisy results, you tend to pick the luckiest one, so I retrained the top three configurations with two more seeds before trusting them (Table 10).

Table 10: Top search configurations retrained with three seeds (validation MAE in Wh)

Configuration	Seed 42	Seed 7	Seed 123	Mean
Search rank 1	22.94	24.70	23.64	23.76
Search rank 2	23.18	24.05	23.43	23.55
Search rank 3	23.28	25.09	23.69	24.02
Default CNN-LSTM	22.96	23.22	22.98	23.05

Over three seeds, all three of the "best" configurations turned out to be **less accurate and less stable** than the default settings I had started with. The higher learning rate and the longer window made training noticeably more sensitive to the starting weights. I therefore **kept the default configuration**. The search did not find a better model, but it did confirm that the default settings were already close to the best available for this data, and that is a useful result too.

7.2 Dropout Experiment

Dropout was the one setting I explored more carefully, as the brief suggests. I trained the CNN-LSTM with dropout rates from 0.0 to 0.4, three seeds each (Table 11, Figure 13).

Table 11: Dropout experiment on the CNN-LSTM, averaged over three seeds

Dropout rate	Val. MAE (Wh)	Spread (std)	Val. RMSE (Wh)
0.0	24.16	0.14	59.19
0.1	23.44	0.21	58.92
0.2	23.05	0.14	58.33
0.3	23.00	0.11	58.23
0.4	23.42	0.19	59.51

Training without dropout was clearly the worst option, which confirms that regularisation helps. Rates of 0.2 and 0.3 were practically tied, and I chose **0.3** because it had the lowest error on both measures.

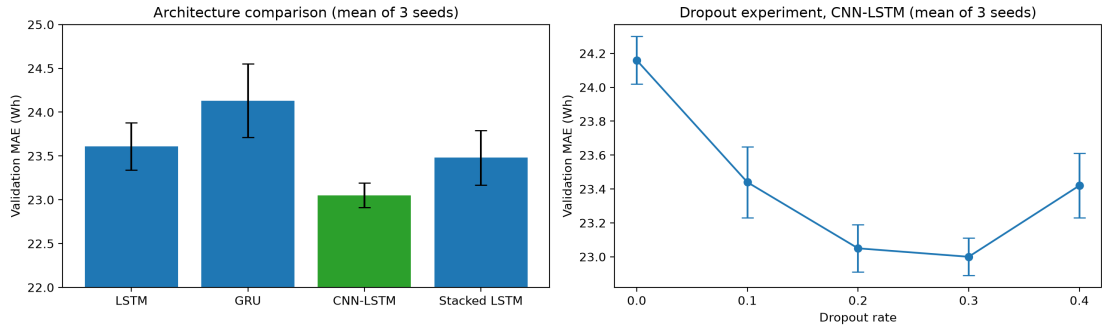


Figure 13: Architecture comparison and dropout experiment, with spread across seeds

7.3 Early Stopping

Every model in this project was trained with early stopping on the validation loss, using a patience of 10 epochs and restoring the best weights, together with halving the learning rate after five epochs without progress. Most runs stopped somewhere between 30 and 60 epochs. This removed the need to guess a fixed number of epochs and protected against both under-training and overfitting.

7.4 Seed Ensemble

For the final model I took the three CNN-LSTMs trained with dropout 0.3 (seeds 42, 7 and 123) and averaged their predictions. Because each network starts from different random weights, they make partly different mistakes, and averaging cancels some of them out. On the validation set this lowered MAE from 23.00 Wh for a typical single network to **22.55 Wh**. The cost is that a prediction now requires three networks instead of one, which is negligible here.

7.5 Performance Before and After Optimisation

Table 12 compares the CNN-LSTM before and after optimisation on the test set.

Table 12: CNN-LSTM test performance before and after optimisation

Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	R^2
Before: single network	23.70	57.55	19.72	0.57
After: 3-network ensemble	22.80	55.71	19.01	0.60
Change	-3.8%	-3.2%	-0.71 pts	+0.03

8 Challenges and Solutions

8.1 A Strong Naive Baseline

At a ten-minute horizon, simply repeating the last reading is hard to beat, and neither Linear Regression nor the Random Forest managed it on MAE. Including the naive forecast from the start made this visible early and gave the deep learning models an honest target. Without it, the Random Forest might have looked like a good model.

8.2 A Skewed Target and Slow Training

My first LSTM, trained on the plain log target, only reached an MAE of 26.8 Wh. The network was spending its first epochs just shifting its output up to the target's average of about 4.3. Once I standardised the log target to zero mean and unit variance, using training statistics only, the same network reached 23.4 Wh. It was the single most effective change in the whole project.

8.3 Avoiding Data Leakage

It is easy to build a forecasting model that accidentally uses information from the interval it is predicting, which produces results that look excellent but cannot be repeated in practice. I shifted all sensor and consumption features by at least one step, split the data by time, fitted the scaler on training data only, and tested the feature pipeline directly to make sure nothing from the future slipped through.

8.4 Telling Real Improvements from Luck

Results moved by up to 1.8 Wh MAE just by changing the random seed, which is more than many of the differences I wanted to measure. I responded by comparing architectures, search results and dropout rates across three seeds rather than one. This is what revealed that the apparent winners of the random search were mostly lucky, and the final seed ensemble reduces this variation directly.

8.5 Reporting Precision

In an early version of the search summary, every column was rounded to three decimal places, which displayed a learning rate of 0.0003 as 0.0. The models themselves were unaffected, but the saved table was misleading. I changed the code to round only the error columns.

8.6 Deploying the Model

To share the model, I published it with an interactive demo on Hugging Face [12]. The hosting environment ran Python 3.10 with an older version of Keras that could not read the saved model's configuration. I wrote a small loader that rebuilds the network from the project's own code and loads only the trained weights. I checked that its predictions match the original model's exactly.

9 Conclusion

9.1 Summary

In this project I built a deep learning model that forecasts a household's appliance energy use ten minutes ahead. After exploring the data, preparing it without leakage and building features guided by autocorrelation, I compared four architectures and tuned the best of them. The final **CNN-LSTM ensemble** reaches an **MAE of 22.80 Wh**, an **RMSE of 55.71 Wh**, a **MAPE of 19.01%** and an R^2 of **0.60** on a later test period. It outperforms every baseline and compares well with published results on the same data, under a stricter form of evaluation.

9.2 Key Findings

- Recent consumption and the time of day hold almost all of the useful information. The temperature and humidity sensors add very little at a ten-minute horizon.
- The biggest improvements came from lag features chosen by autocorrelation, from the log and standardised target, and from averaging several networks. Extra depth and random-search tuning did not help.
- Comparing models across several random seeds was essential for separating real progress from chance.

9.3 Limitations

The model's weak point is the sudden spike. A spike usually starts when someone switches on an appliance, and there is rarely any sign of it in the preceding readings. The model therefore reacts one step late and underestimates the peak, and most of its remaining error comes from these moments. It was also trained on a single house over four and a half months, so it would need retraining before being used in another home or another season.

9.4 Future Work

- Extend the model to forecast further ahead, for example one hour or a full day.
- Add information that could signal a spike before it happens, such as occupancy or appliance-level readings.
- Predict the probability of a spike with a separate model and combine it with the forecast.
- Train on longer periods and more households to capture seasonal and behavioural differences.

The trained model [11] and an interactive demo that replays the test period [12] are publicly available, and the complete code is in the project repository [10].

References

- [1] L. M. Candanedo, V. Feldheim and D. Deramaix, “Data driven prediction models of energy use of appliances in a low-energy house,” *Energy and Buildings*, vol. 140, pp. 81–97, 2017.
- [2] L. M. Candanedo, “Appliances Energy Prediction [Dataset],” UCI Machine Learning Repository, 2017. Available: <https://archive.ics.uci.edu/dataset/374/appliances+energy+prediction>
- [3] J. Runge and R. Zmeureanu, “A review of deep learning techniques for forecasting energy use in buildings,” *Energies*, vol. 14, no. 3, p. 608, 2021.
- [4] I. W. A. Suranata, I. N. K. Wardana, N. Jawas and I. K. A. A. Aryanto, “Feature engineering and long short-term memory for energy use of appliances prediction,” *TELKOMNIKA*, vol. 19, no. 3, pp. 920–930, 2021.
- [5] C. F. Assadian and F. Assadian, “Data-driven modeling of appliance energy usage,” *Energies*, vol. 16, no. 22, p. 7536, 2023.
- [6] T.-Y. Kim and S.-B. Cho, “Predicting residential energy consumption using CNN-LSTM neural networks,” *Energy*, vol. 182, pp. 72–81, 2019.
- [7] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [8] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [9] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.
- [10] Project source code, “Multivariate energy forecasting.” Available: <https://github.com/NethranjaliSE/multivariate-energy-forecasting>
- [11] Trained model, “Appliance energy CNN-LSTM ensemble.” Available: <https://huggingface.co/NethranjaliSE/appliance-energy-cnn-lstm>
- [12] Interactive demo, “Appliance energy forecasting.” Available: <https://huggingface.co/spaces/NethranjaliSE/appliance-energy-forecasting>